new idea

CREATING tables

Creating Tables

```
CREATE TABLE teacher (
    id VARCHAR(5),
    name VARCHAR(20),
    dept_name VARCHAR(20),
    salary NUMERIC(8,2)
);
```

SQL is **case insensitive**

statements end with ;

attribute names unique within table

Advice for table and attribute names :

first character should be alphabetical

use _ to separate words

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Domain Types

`varchar` (N): variable-length string of **at most** N characters

`numeric` (P, D): number with P total digits, D digits to the right of decimal place

`char` (N): fixed-length string of N characters

`int`: 32 bits `smallint`: 16 bits `tinyint`: 8 bits `bigint`: 64 bits

`float` (N): floating point value with at least N bits

`date`, `time`, `text`, ...

every attribute can take the **NULL** value

Adding Data

newly created tables are empty

```
CREATE TABLE teacher (
    id VARCHAR(5),
    name VARCHAR(20),
    dept_name VARCHAR(20),
    salary NUMERIC(8,2)
);
```

to insert a row :

```
INSERT INTO teacher
VALUES ( '10101', 'Srinivasan', 'Comp.  Sci.', 65000 );
```

string values are with single quotes

values appear in the same order as **CREATE TABLE**

insert a row by attributes:

```
INSERT INTO teacher ( id, name, salary )
VALUES ( '4234', 'JustaTest', 50000 );
```

default value of VARCHAR(20) : **NULL**

```
4234 | JustaTest |  | 50000.00
```

new idea

DELETING tables

Deleting

delete some rows

```
DELETE FROM table WHERE condition
```

```
DELETE FROM teacher WHERE id = '4234' ;
```

```
DELETE FROM teacher WHERE salary <= (SELECT avg(salary) FROM teacher) ;
```

delete all rows (so the table is empty)

```
DELETE FROM teacher ;
```

delete even the table

```
DROP TABLE teacher ;
```

delete the entire database

```
DROP DATABASE universityDB ;
```

teacher

id	*name*	*dept_name*	*salary*
83821	Brandt	Comp. Sci.	92000.00
58583	Califieri	History	62000.00
76766	Crick	Biology	72000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
98345	Kim	Elec. Eng.	80000.00
15151	Mozart	Music	40000.00
76543	Singh	Finance	80000.00
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00



avg(salary) : 74833.33

id	*name*	*dept_name*	*salary*
12121	Wu	Finance	90000.00
22222	Einstein	Physics	95000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
76543	Singh	Finance	80000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

More Data Types

VARCHAR(N) has limited size, however we have

TEXT : use for larger text, for KB or MB of text data

to store images, binary files, all sorts of data

BLOB : Binary Large OBject

to store dates, times we can use

TIMESTAMP DATE TIME INTERVAL

Table 8.9. Date/Time Types

Name	Storage Size	Description	Low Value	High Value	Resolution
timestamp [(p)] [without time zone]	8 bytes	both date and time (no time zone)	4713 BC	294276 AD	1 microsecond / 14 digits
timestamp [(p)] with time zone	8 bytes	both date and time, with time zone	4713 BC	294276 AD	1 microsecond / 14 digits
date	4 bytes	date (no time of day)	4713 BC	5874897 AD	1 day
time [(p)] [without time zone]	8 bytes	time of day (no date)	00:00:00	24:00:00	1 microsecond / 14 digits
time [(p)] with time zone	12 bytes	times of day only, with time zone	00:00:00+1459	24:00:00-1459	1 microsecond / 14 digits
interval [fields] [(p)]	16 bytes	time interval	-178000000 years	178000000 years	1 microsecond / 14 digits

Example: DATE

```
CREATE TABLE mytable (
    id VARCHAR(5),
    name VARCHAR(20),
    dept_name VARCHAR(20),
    dateofbirth DATE
);
```

```
INSERT INTO mytable
VALUES ( '10101', 'Srinivasan', 'Comp.  Sci.', '02-10-1990' );
```

```
myuniversitydb-# INSERT INTO mytable
myuniversitydb-# VALUES ('10101', 'Srinivasan', 'Comp. Sci.', '02-10-1990');
INSERT 0 1
```

can use various functions in **SQL** for different types

```
myuniversitydb=# SELECT EXTRACT (YEAR FROM dateofbirth)
myuniversitydb-# FROM mytable;
 extract
---------
    1990
(1 row)
```

Converting between types

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT salary / 1000
FROM teacher;
```

?column?
6.5000000000000000
9.0000000000000000
4.0000000000000000
9.5000000000000000
6.0000000000000000
8.7000000000000000
7.5000000000000000
6.2000000000000000
8.0000000000000000
7.2000000000000000
9.2000000000000000
8.0000000000000000

```
SELECT salary::int / 1000
FROM teacher;
```

?column?
6
9
4
9
6
8
7
6
8
7
9
8

Many functions available

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT 'my name is: ' || name
FROM teacher;
```

?column?
my name is: Califieri
my name is: Singh
my name is: Kim
my name is: Srinivasan
my name is: Wu
my name is: Mozart
my name is: Einstein
my name is: El Said
my name is: Gold
my name is: Katz
my name is: Crick
my name is: Brandt

new idea

UPDATING tables

UPDATE

Goal: add 10k to salary if salary less than 70k

```
UPDATE teacher
   SET salary = salary + 10000
WHERE salary <= 70000;
```

```
UPDATE table_name
   SET attr1 = val1,
       attr2 = val2,
       attr3 = val3, ...
WHERE condition;
```

WHERE is optional
can contain nested queries

teacher

id	*name*	*dept_name*	*salary*
83821	Brandt	Comp. Sci.	92000.00
58583	Califieri	History	62000.00
76766	Crick	Biology	72000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
98345	Kim	Elec. Eng.	80000.00
15151	Mozart	Music	40000.00
76543	Singh	Finance	80000.00
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00



id	*name*	*dept_name*	*salary*
83821	Brandt	Comp. Sci.	92000.00
58583	Califieri	History	72000.00
76766	Crick	Biology	72000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	70000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
98345	Kim	Elec. Eng.	80000.00
15151	Mozart	Music	50000.00
76543	Singh	Finance	80000.00
10101	Srinivasan	Comp. Sci.	75000.00
12121	Wu	Finance	90000.00

Alter Table

```
ALTER TABLE teacher ADD COLUMN age NUMERIC(3,0);
```

```
ALTER TABLE teacher DROP COLUMN age;
```

```
ALTER TABLE teacher RENAME COLUMN name TO nom;
```

```
ALTER TABLE teacher RENAME TO teacherUniversity;
```

```
ALTER TABLE teacher ADD CONSTRAINT salarycheck CHECK(salary < 100000);
```

```
ALTER TABLE teacher DROP CONSTRAINT salarycheck;
```

```
ALTER TABLE [ ONLY ] table [ * ]
    ADD [ COLUMN ] [ column type ] [ column_constraint [ ... ] ]
ALTER TABLE [ ONLY ] table [ * ]
    ALTER [ COLUMN ] column [ SET DEFAULT value | DROP DEFAULT ]
ALTER TABLE [ ONLY ] table [ * ]
    ALTER [ COLUMN ] column SET STATISTICS integer
ALTER TABLE [ ONLY ] table [ * ]
    RENAME [ COLUMN ] column TO newcolumn
ALTER TABLE table
    RENAME TO new_table
ALTER TABLE table
    ADD table_constraint_definition
ALTER TABLE [ ONLY ] table
    DROP CONSTRAINT constraint [ RESTRICT | CASCADE ]
ALTER TABLE table
    OWNER TO new_owner
```

many other utilities: learn and use as you need them...

```
yuniversitycbd=# \h
Available help:
  ABORT                            ALTER TEXT SEARCH TEMPLATE       CREATE PUBLICATION               DROP FUNCTION                    IMPORT FOREIGN SCHEMA
  ALTER AGGREGATE                  ALTER TRIGGER                    CREATE ROLE                      DROP GROUP                       INSERT
  ALTER COLLATION                  ALTER TYPE                       CREATE RULE                      DROP INDEX                       LISTEN
  ALTER CONVERSION                 ALTER USER                       CREATE SCHEMA                    DROP LANGUAGE                    LOAD
  ALTER DATABASE                   ALTER USER MAPPING               CREATE SEQUENCE                  DROP MATERIALIZED VIEW           LOCK
  ALTER DEFAULT PRIVILEGES         ALTER VIEW                       CREATE SERVER                    DROP OPERATOR                    MOVE
  ALTER DOMAIN                     ANALYZE                          CREATE STATISTICS                DROP OPERATOR CLASS              NOTIFY
  ALTER EVENT TRIGGER              BEGIN                            CREATE SUBSCRIPTION              DROP OPERATOR FAMILY             PREPARE
  ALTER EXTENSION                  CALL                             CREATE TABLE                     DROP OWNED                       PREPARE TRANSACTION
  ALTER FOREIGN DATA WRAPPER       CHECKPOINT                       CREATE TABLE AS                  DROP POLICY                      REASSIGN OWNED
  ALTER FOREIGN TABLE              CLOSE                            CREATE TABLESPACE                DROP PROCEDURE                   REFRESH MATERIALIZED VIEW
  ALTER FUNCTION                   CLUSTER                          CREATE TEXT SEARCH CONFIGURATION DROP PUBLICATION                 REINDEX
  ALTER GROUP                      COMMENT                          CREATE TEXT SEARCH DICTIONARY    DROP ROLE                        RELEASE SAVEPOINT
  ALTER INDEX                      COMMIT                           CREATE TEXT SEARCH PARSER        DROP ROUTINE                     RESET
  ALTER LANGUAGE                   COMMIT PREPARED                  CREATE TEXT SEARCH TEMPLATE      DROP RULE                        REVOKE
  ALTER LARGE OBJECT               COPY                             CREATE TRANSFORM                 DROP SCHEMA                      ROLLBACK
  ALTER MATERIALIZED VIEW          CREATE ACCESS METHOD             CREATE TRIGGER                   DROP SEQUENCE                    ROLLBACK PREPARED
  ALTER OPERATOR                   CREATE AGGREGATE                 CREATE TYPE                      DROP SERVER                      ROLLBACK TO SAVEPOINT
  ALTER OPERATOR CLASS             CREATE CAST                      CREATE USER                      DROP STATISTICS                  SAVEPOINT
  ALTER OPERATOR FAMILY            CREATE COLLATION                 CREATE USER MAPPING              DROP SUBSCRIPTION                SECURITY LABEL
  ALTER POLICY                     CREATE CONVERSION                CREATE VIEW                      DROP TABLE                       SELECT
  ALTER PROCEDURE                  CREATE DATABASE                  DEALLOCATE                       DROP TABLESPACE                  SELECT INTO
  ALTER PUBLICATION                CREATE DOMAIN                    DECLARE                          DROP TEXT SEARCH CONFIGURATION   SET
  ALTER ROLE                       CREATE EVENT TRIGGER             DELETE                           DROP TEXT SEARCH DICTIONARY      SET CONSTRAINTS
  ALTER ROUTINE                    CREATE EXTENSION                 DISCARD                          DROP TEXT SEARCH PARSER          SET ROLE
  ALTER RULE                       CREATE FOREIGN DATA WRAPPER      DO                               DROP TEXT SEARCH TEMPLATE        SET SESSION AUTHORIZATION
  ALTER SCHEMA                     CREATE FOREIGN TABLE             DROP ACCESS METHOD               DROP TRANSFORM                   SET TRANSACTION
  ALTER SEQUENCE                   CREATE FUNCTION                  DROP AGGREGATE                   DROP TRIGGER                     SHOW
  ALTER SERVER                     CREATE GROUP                     DROP CAST                        DROP TYPE                        START TRANSACTION
  ALTER STATISTICS                 CREATE INDEX                     DROP COLLATION                   DROP USER                        TABLE
  ALTER SUBSCRIPTION               CREATE LANGUAGE                  DROP CONVERSION                  DROP USER MAPPING                TRUNCATE
  ALTER SYSTEM                     CREATE MATERIALIZED VIEW         DROP DATABASE                    DROP VIEW                        UNLISTEN
  ALTER TABLE                      CREATE OPERATOR                  DROP DOMAIN                      END                              UPDATE
  ALTER TABLESPACE                 CREATE OPERATOR CLASS            DROP EVENT TRIGGER               EXECUTE                          VACUUM
  ALTER TEXT SEARCH CONFIGURATION  CREATE OPERATOR FAMILY           DROP EXTENSION                   EXPLAIN                          VALUES
  ALTER TEXT SEARCH DICTIONARY     CREATE POLICY                    DROP FOREIGN DATA WRAPPER        FETCH                            WITH
  ALTER TEXT SEARCH PARSER         CREATE PROCEDURE                 DROP FOREIGN TABLE               GRANT
yuniversitycbd=#
```

new idea

SUPERKEY

CANDIDATE KEY

Keys

Recall: duplicate tuples are not allowed

no two tuples with the *same* Account_id, Branch_name and Amount.

sometimes even something stronger is true:

no two tuples with the *same* Account_id

A Superkey: combination of attributes present **at most once** in the table

in other words, knowing their value $\Longrightarrow$ all other attribute values

Account_id	Branch_name	Amount
49234	Paris	334.6
83123	Lille	231.0
34234	Marseille	334.6
62433	Tour	391
31412	Nice	323.2

Superkeys

✓ course_id

✗ title

✓ course_id, title

✓ title, dept_name

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

course

Superkeys

If something is a superkey, then adding more attributes to it, keeps it a superkey?

YES

✓ course_id

automatically implies that these are also superkeys:

course_id, title ✓

course_id, credits ✓

course_id, title, credits ✓

course_id, . . . ✓

Superkeys

If something is a superkey, then **removing** attributes keeps a superkey?

✓ course_id, dept_name ✗ dept_name

NO

✗ course_id, dept_name

✓ title, dept_name

✓ course_id

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

course

Candidate Key

candidate keys: all *minimal* superkeys

↓

superkey AND removing *any* attribute makes it **NOT** a superkey

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

course

Candidate keys of **course**:

✗ course_id, dept_name

✗ dept_name

✓ course_id

✗ dept_name, credits

Superkeys

- ✓ Account_id, Branch_name, Amount
- ✓ Account_id, Branch_name
- ✓ Account_id, Amount
- ✓ Account_id

there can be two lines with same Branch_name and same Amount.

- ✗ Branch_name, Amount
- ✗ Branch_name
- ✗ Amount

Account_id	Branch_name	Amount
49234	Paris	334.6
83123	Lille	231.0
34234	Marseille	334.6
62433	Tour	391
31412	Nice	323.2

new idea

PRIMARY KEY

KEYS

primary key: one of the candidate keys that you choose

indicated by underlining it

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

only a single primary key possible for each table

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

teacher

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

course

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

section

id	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

teaches

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

department

Keys

dept_name is a foreign key in course, referencing department

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

course

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

department

Obligatory: each foreign key value **must** exist in primary key table

as we will see later: makes maintaining *consistency* between tables more complicated

```
myuniversitydb=# create table FIRST ( a text, b text, primary key(a) );
CREATE TABLE
myuniversitydb=# create table SECOND ( c text, d text, foreign key(c) references FIRST(a) );
CREATE TABLE
```

```
myuniversitydb=# create table FIRST ( a text, b text, primary key(a) );
CREATE TABLE
myuniversitydb=# create table SECOND ( c text, d text, foreign key(c) references FIRST(b) );
ERROR:  there is no unique constraint matching given keys for referenced table "first"
```

```
myuniversitydb=# create table FIRST ( a text, b text );
CREATE TABLE
myuniversitydb=# create table SECOND ( c text, d text, foreign key(c) references FIRST(a) );
ERROR:  there is no unique constraint matching given keys for referenced table "first"
```

new idea

CONSTRAINT : PRIMARY KEY

Primary Key

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
CREATE TABLE teacher (
  id   VARCHAR(5),
  name   VARCHAR(20),
  dept_name  VARCHAR(20),
  salary   NUMERIC(8,2),
  PRIMARY KEY (id)
);
```

PRIMARY KEY cannot have **null** value

No two rows with same value of primary key

```
myuniversitydb=# INSERT INTO teacher VALUES (58583, 'MyName', 'Comp. Sci.', 100000);
ERROR:  duplicate key value violates unique constraint "teacher_pkey"
DETAIL:  Key (id)=(58583) already exists.
```

Primary Key

```
CREATE TABLE section (
  course_id  VARCHAR(8),
  sec_id   VARCHAR(8),
  semester  VARCHAR(6),
  year   NUMERIC(4,0),
  building  VARCHAR(15),
  room_number  VARCHAR(7),
  time_slot_id  VARCHAR(4),
  PRIMARY KEY (course_id, sec_id, semester, year)
);
```



no two rows with same value of `course_id` *and* `sec_id` *and* `semester` *and* `year`

Group By

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT name
FROM teacher
GROUP BY id;
```



name
Brandt
Singh
Gold
Califieri
Einstein
Crick
Wu
Kim
Mozart
El Said
Srinivasan
Katz

```
SELECT id
FROM teacher
GROUP BY name;
```



```
ERROR:  column "teacher.id" must appear in the GROUP BY
clause or be used in an aggregate function

LINE 1: SELECT id
```

Not an error:
PRIMARY KEY `id` value fixes `name`
each mini-table with same `id` has same `name`

new idea

CONSTRAINT :

NOT NULL

UNIQUE

Default Values

for each attribute, the default value is automatically set to **null**

can use the DEFAULT keyword to specify another default value

```
CREATE TABLE teacher (
  id   VARCHAR(5),
  name   VARCHAR(20) NOT NULL,
  dept_name  VARCHAR(20) DEFAULT 'Music',
  salary   NUMERIC(8,2)
);
```

```
universitydb=# INSERT INTO teacher (id,name,salary)
 VALUES('98313','TestName', 30);
INSERT 0 1
universitydb=# SELECT * FROM teacher;
  id   |     name     | dept_name  |  salary
-------+--------------+------------+----------
 10101 | Srinivasan   | Comp. Sci. | 65000.00
 12121 | Wu           | Finance    | 90000.00
 15151 | Mozart       | Music      | 40000.00
 22222 | Einstein     | Physics    | 95000.00
 32343 | El Said      | History    | 60000.00
 33456 | Gold         | Physics    | 87000.00
 45565 | Katz         | Comp. Sci. | 75000.00
 58583 | Califieri    | History    | 62000.00
 76543 | Singh        | Finance    | 80000.00
 76766 | Crick        | Biology    | 72000.00
 83821 | Brandt       | Comp. Sci. | 92000.00
 98345 | Kim          | Elec. Eng. | 80000.00
 98313 | TestName     | Music      |    30.00
(13 rows)
```

Primary Key

every attribute can take the **null** value

as we saw, PRIMARY KEY cannot have **null** value

sometimes we never want an attribute to take **null** value

```
CREATE TABLE teacher (
  id   VARCHAR(5),
  name   VARCHAR(20) NOT null,
  dept_name  VARCHAR(20),
  salary   NUMERIC(8,2),
  PRIMARY KEY (id)
);
```

```
universitydb=# INSERT INTO teacher VALUES ('34123', null, 'Music', 50000);
ERROR:  null value in column "name" violates not-null constraint
DETAIL:  Failing row contains (34123, null, Music, 50000.00).
```

UNIQUE constraint

UNIQUE
can specify other combination of attributes to be unique

recall: this was the notion of superkey

```
CREATE TABLE teacher (
  id   VARCHAR(5),
  name   VARCHAR(20) NOT NULL,
  dept_name  VARCHAR(20),
  salary   NUMERIC(8,2),
  PRIMARY KEY (id),
  UNIQUE (name, salary)
);
```

```
universitydb=# INSERT INTO teacher VALUES ('34131', 'TestName', 'Music', 50000);
INSERT 0 1
universitydb=# INSERT INTO teacher VALUES ('84524', 'TestName', 'History', 50000);
ERROR:  duplicate key value violates unique constraint "teacher_name_salary_key"
DETAIL:  Key (name, salary)=(TestName, 50000.00) already exists.
```

UNIQUE CONSTRAINT

```
CREATE TABLE teacher (
  id   VARCHAR(5),
  name   VARCHAR(20) NOT NULL,
  dept_name  VARCHAR(20),
  salary   NUMERIC(8,2),
  PRIMARY KEY (id),
  UNIQUE (name, salary)
);
```

unlike PRIMARY KEY, UNIQUE does not exclude **null**

considers two **null** values to be different

can have many identical rows with **null** values !

```
universitydb=# INSERT INTO teacher VALUES ('84524', 'TestName', 'History', null);
INSERT 0 1
universitydb=# INSERT INTO teacher VALUES ('62342', 'TestName', 'History', null);
INSERT 0 1
```

advice: usually specify NOT null for all attributes in UNIQUE constraint

new idea

CONSTRAINT :

CHECK

CHECK CONSTRAINT

can specify arbitrary condition on attribute values using CHECK constraints

```
CREATE TABLE teacher (
  id   VARCHAR(5),
  name   VARCHAR(20) NOT NULL,
  dept_name  VARCHAR(20),
  salary   NUMERIC(8,2),
  PRIMARY KEY (id),
  CHECK (salary <= 100000)
);
```

```
universitydb=# INSERT INTO teacher VALUES ('34341', 'TestName', 'Music', 120000);
ERROR:  new row for relation "teacher" violates check constraint "teacher_salary_ch
eck1"
DETAIL:  Failing row contains (34341, TestName, Music, 120000.00).
```

CHECK CONSTRAINT

can specify arbitrary condition on attribute values using CHECK constraints

```
CREATE TABLE teacher (
  id   VARCHAR(5),
  name   VARCHAR(20) NOT NULL,
  dept_name  VARCHAR(20),
  salary   NUMERIC(8,2),
  PRIMARY KEY (id),
  CHECK (dept_name IN ('Comp. Sci.', 'Music', 'Finance', 'History'))
);
```

```
universitydb=# INSERT INTO teacher VALUES ('41234', 'TestName', 'Math', '60000');
ERROR:  new row for relation "teacher" violates check constraint "teacher_dept_name
 check"
DETAIL:  Failing row contains (41234, TestName, Math, 60000.00).
```

new idea

CONSTRAINT :

REFERENTIAL INTEGRITY

recall

KEYS

dept_name is a foreign key in **course**, referencing **department**



course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

course

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

department

Obligatory: each foreign key value **must** exist in primary key table

as we will see later: makes maintaining *consistency* between tables more complicated

Foreign Key Constraint

```
CREATE TABLE teacher (
  id   VARCHAR(5),
  name   VARCHAR(20) NOT NULL,
  dept_name  VARCHAR(20),
  salary   NUMERIC(8,2),
  PRIMARY KEY (id),
  FOREIGN KEY (dept_name) REFERENCES department
);
```

FOREIGN KEY constraint does *not* imply NOT null for **dept_name** in this table

However, each non-null foreign key value **must** exist in primary key table

each **dept_name** in teacher table must be in department table

(except if the value is **null**)

Foreign Key Constraint

Types of errors that can happen with foreign key constraints :

Error 1: add row where FOREIGN KEY value not present in referenced relation

For example, adding person with department 'Philosophy'

```
universitydb=# INSERT INTO teacher VALUES ('3431','TestName','Philosophy',60000);
ERROR:  insert or update on table "teacher" violates foreign key constraint "teache
r_dept_name_fkey"
DETAIL:  Key (dept_name)=(Philosophy) is not present in table "department".
```

```
CREATE TABLE teacher (
  id   VARCHAR(5),
  name   VARCHAR(20) NOT NULL,
  dept_name  VARCHAR(20),
  salary   NUMERIC(8,2),
  PRIMARY KEY (id),
  FOREIGN KEY (dept_name) REFERENCES department
);
```

Foreign Key Constraint

Types of errors that can happen with foreign key constraints :

Error 2: deleting a PRIMARY KEY in the referenced relation

For example, deleting department 'History' from the department table

```
universitydb=# DELETE FROM department WHERE dept_name = 'History';
ERROR:  update or delete on table "department" violates foreign key constraint "tea
cher_dept_name_fkey" on table "teacher"
DETAIL:  Key (dept_name)=(History) is still referenced from table "teacher".
```

```
CREATE TABLE teacher (
  id   VARCHAR(5),
  name   VARCHAR(20) NOT NULL,
  dept_name  VARCHAR(20),
  salary   NUMERIC(8,2),
  PRIMARY KEY (id),
  FOREIGN KEY (dept_name) REFERENCES department
);
```

Handling Error 2

Error 2: deleting a PRIMARY KEY in the referenced relation

Solution 1: disallow such operation!

This is the default if nothing is specified

```
universitydb=# DELETE FROM department WHERE dept_name = 'History';
ERROR:  update or delete on table "department" violates foreign key constraint "tea
cher_dept_name_fkey" on table "teacher"
DETAIL:  Key (dept_name)=(History) is still referenced from table "teacher".
```

```
CREATE TABLE teacher (
  id   VARCHAR(5),
  name   VARCHAR(20) NOT NULL,
  dept_name  VARCHAR(20),
  salary   NUMERIC(8,2),
  PRIMARY KEY (id),
  FOREIGN KEY (dept_name) REFERENCES department
);
```

Handling Error 2

Error 2: deleting a PRIMARY KEY in the referenced relation

Solution 2: allow delete but set all referencing values to **null**

```
universitydb=# DELETE FROM department WHERE dept_name = 'History';
DELETE 1
universitydb=# SELECT * FROM teacher;
  id   |    name    | dept_name  |  salary
-------+------------+------------+----------
 10101 | Srinivasan | Comp. Sci. | 65000.00
 12121 | Wu         | Finance    | 90000.00
 15151 | Mozart     | Music      | 40000.00
 22222 | Einstein   | Physics    | 95000.00
 33456 | Gold       | Physics    | 87000.00
 45565 | Katz       | Comp. Sci. | 75000.00
 76543 | Singh      | Finance    | 80000.00
 76766 | Crick      | Biology    | 72000.00
 83821 | Brandt     | Comp. Sci. | 92000.00
 98345 | Kim        | Elec. Eng. | 80000.00
 32343 | El Said    |            | 60000.00
 58583 | Califieri  |            | 62000.00
(12 rows)
```

```
CREATE TABLE teacher (
  id   VARCHAR(5),
  name   VARCHAR(20) NOT NULL,
  dept_name  VARCHAR(20),
  salary   NUMERIC(8,2),
  PRIMARY KEY (id),
  FOREIGN KEY (dept_name) REFERENCES department
                ON DELETE SET NULL
);
```

Handling Error 2

Error 2: deleting a PRIMARY KEY in the referenced relation

Solution 3: propagate the delete operation—called **CASCADE**

when row deleted from department delete all rows in teacher that use that value

```
CREATE TABLE teacher (
  id   VARCHAR(5),
  name   VARCHAR(20) NOT NULL,
  dept_name  VARCHAR(20),
  salary   NUMERIC(8,2),
  PRIMARY KEY (id),
  FOREIGN KEY (dept_name) REFERENCES department
                ON DELETE CASCADE
);
```

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
universitydb=# DELETE FROM department WHERE dept_name = 'History';
DELETE 1
universitydb=# SELECT * FROM teacher;
  id   |    name    | dept_name  |  salary
-------+------------+------------+----------
 10101 | Srinivasan | Comp. Sci. | 65000.00
 12121 | Wu         | Finance    | 90000.00
 15151 | Mozart     | Music      | 40000.00
 22222 | Einstein   | Physics    | 95000.00
 33456 | Gold       | Physics    | 87000.00
 45565 | Katz       | Comp. Sci. | 75000.00
 76543 | Singh      | Finance    | 80000.00
 76766 | Crick      | Biology    | 72000.00
 83821 | Brandt     | Comp. Sci. | 92000.00
 98345 | Kim        | Elec. Eng. | 80000.00
(10 rows)
```

Naming Constraints

each constraint has a name

so database system can tell you which constraint causes error

There is a default name, but you can assign the name yourself

```
CREATE TABLE teacher (
  id   VARCHAR(5),
  name   VARCHAR(20) NOT NULL,
  dept_name  VARCHAR(20),
  salary   NUMERIC(8,2),
  CONSTRAINT myprimarykey PRIMARY KEY (id),
  CONSTRAINT myfirstforeignkey
    FOREIGN KEY (dept_name) REFERENCES department
      ON DELETE CASCADE
);
```

```
universitydb=# INSERT INTO teacher VALUES ('32343', 'TestName', 'Music', 50000);
ERROR:  duplicate key value violates unique constraint "myprimarykey"
DETAIL:  Key (id)=(32343) already exists.
```

new idea

UNIVERSITY TABLES

course

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4

```
create table course (
   course_id  varchar(8),
   title   varchar(50),
   dept_name  varchar(20),
   credits  numeric(2,0),
   primary key (course_id),
   foreign key (dept_name) references department
 );
```

MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

```
varchar(8) :  variable-length string of at-most 8 characters
numeric(2, 0) :  number, 2 total digits, out of which 0 for decimal
```

(from Database System Concepts)

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00

```
create table teacher (
  id   varchar(5),
  name   varchar(20),
  dept_name  varchar(20),
  salary   numeric(8,2),
  primary key (id),
  foreign key (dept_name) references department
 );
```

83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

(from Database System Concepts)

```
create table teacher (
  id   varchar(5),
  name   varchar(20),
  dept_name  varchar(20),
  salary   numeric(8,2),
  primary key (id),
  foreign key (dept_name) references department
 );
```

```
myuniversitydb-# \d teacher;
                    Table "public.teacher"
  Column   |         Type          | Collation | Nullable | Default
-----------+-----------------------+-----------+----------+---------
 id        | character varying(5)  |           | not null |
 name      | character varying(20) |           |          |
 dept_name | character varying(20) |           |          |
 salary    | numeric(8,2)          |           |          |
Indexes:
    "teacher_pkey" PRIMARY KEY, btree (id)
Foreign-key constraints:
    "teacher_dept_name_fkey" FOREIGN KEY (dept_name) REFERENCES department(dept_name)
Referenced by:
    TABLE "advisor" CONSTRAINT "advisor_i_id_fkey" FOREIGN KEY (i_id) REFERENCES teacher(id)
    TABLE "teaches" CONSTRAINT "teaches_id_fkey" FOREIGN KEY (id) REFERENCES teacher(id)
```

student

id	*name*	*dept_name*	*tot_cred*
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

```
create table student (
  id varchar(5),
  name varchar(20),
  dept_name varchar(20),
  tot_cred numeric(3,0),
  primary key (id),
  foreign key (dept_name) references department
);
```

(from Database System Concepts)

section

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A

```
create table section (
  course_id  varchar(8),
  sec_id   varchar(8),
  semester  varchar(6),
  year   numeric(4,0),
  building  varchar(15),
  room_number  varchar(7),
  time_slot_id  varchar(4),
  primary key (course_id, sec_id, semester, year),
  foreign key (course_id) references course
 );
```

(from Database System Concepts)

teaches

id	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009

```
create table teaches (
  id   varchar(5),
  course_id  varchar(8),
  sec_id   varchar(8),
  semester  varchar(6),
  year   numeric(4,0),
  primary key (id, course_id, sec_id, semester, year),
  foreign key (course_id, sec_id, semester, year)
                              references section,
  foreign key (id) references teacher
 );
```

98345	EE-181	1	Spring	2009

(from Database System Concepts)

department

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00

```
create table department (
  dept_name  varchar(20),
  building  varchar(15),
  budget numeric(12,2),
  primary key (dept_name)
 );
```

(from Database System Concepts)

takes

id	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

```
create table takes (
  id varchar(5),
  course_id varchar(8),
  sec_id varchar(8),
  semester varchar(6),
  year numeric(4,0),
  grade varchar(2),
  primary key (id, course_id, sec_id, semester, year),
  foreign key (course_id,sec_id, semester, year) references section,
  foreign key (id) references student
);
```

(from Database System Concepts)

advisor

s_id	*i_id*
00128	45565
12345	10101
23121	76543
44553	22222

```
create table advisor (
  s_ID varchar(5),
  i_ID varchar(5),
  primary key (s_ID),
  foreign key (i_ID) references teacher (ID),
  foreign key (s_ID) references student (ID)
);
```

(from Database System Concepts)

Relational Database : University

course

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

```
create table course (
   course_id  varchar(8),
   title   varchar(50),
   dept_name  varchar(20),
   credits  numeric(2,0),
   primary key (course_id),
   foreign key (dept_name) references department
 );
```

section

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

```
create table section (
  course_id  varchar(8),
  sec_id   varchar(8),
  semester  varchar(6),
  year   numeric(4,0),
  building  varchar(15),
  room_number  varchar(7),
  time_slot_id  varchar(4),
  primary key (course_id, sec_id, semester, year),
  foreign key (course_id) references course
 );
```

teaches

id	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

```
create table teaches (
  id   varchar(5),
  course_id  varchar(8),
  sec_id   varchar(8),
  semester  varchar(6),
  year   numeric(4,0),
  primary key (id, course_id, sec_id, semester, year),
  foreign key (course_id,sec_id, semester, year) references section,
  foreign key (id) references teacher
 );
```

department

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

```
create table department (
  dept_name  varchar(20),
  building  varchar(15),
  budget numeric(12,2),
  primary key (dept_name)
 );
```

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
create table teacher (
  id   varchar(5),
  name   varchar(20),
  dept_name  varchar(20),
  salary   numeric(8,2),
  primary key (id),
  foreign key (dept_name) references department
 );
```

new idea

DATABASE SCHEMA

Schema

```
classroom (building, room_number, capacity)

department (dept_name, building, budget)

course (course_id, title, dept_name, credits)

instructor (ID, name, dept_name, salary)

section (course_id, sec_id, semester, year, building, room_number)

teaches (ID, course_id, sec_id, semester, year)

student (ID, name, dept_name, tot_cred)

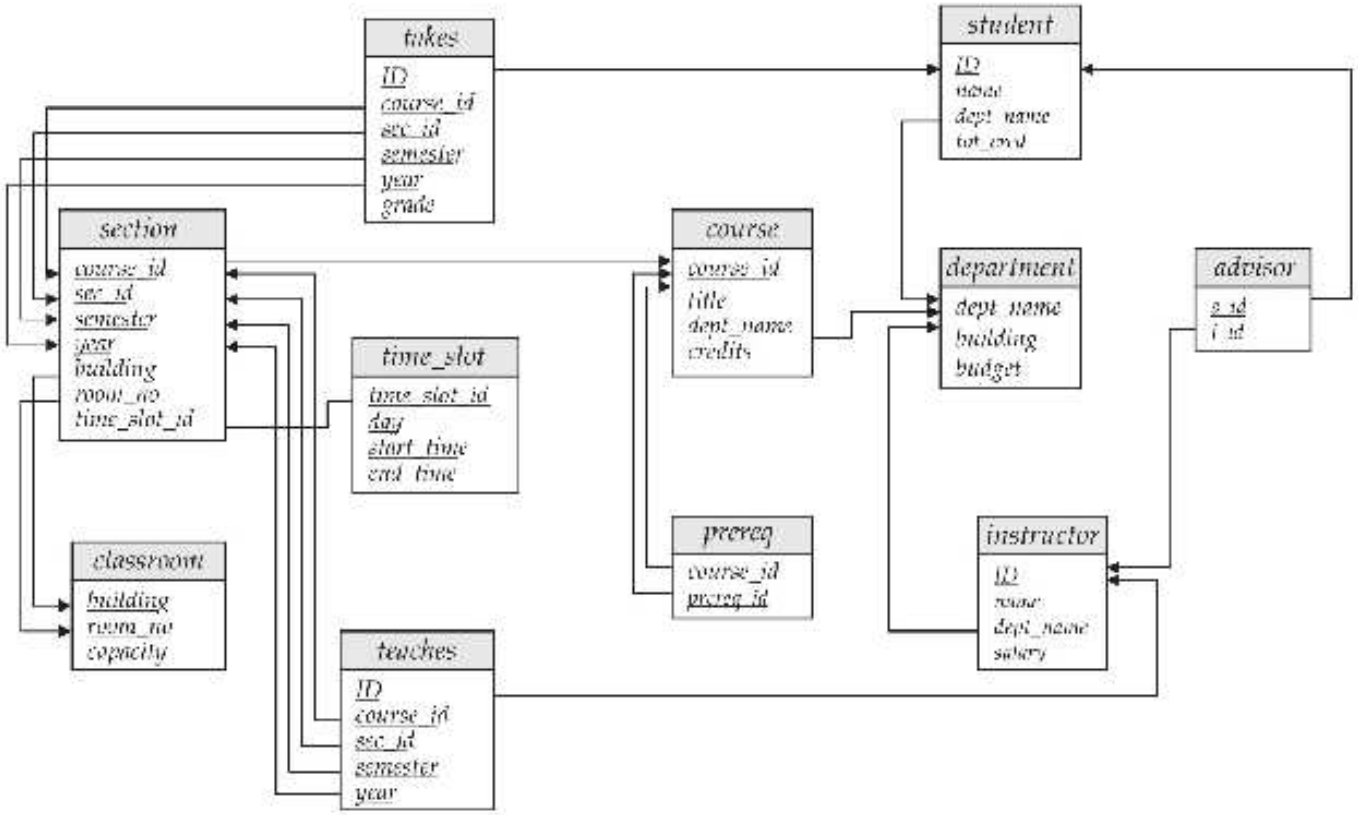
takes (ID, course_id, sec_id, semester, year, grade)

advisor (s_ID, i_ID)

time_slot (time_slot_id, day, start_time, end_time)

prereq (course_id, prereq_id)
```

Schema diagram



new idea

VIEWS

Saving Query to Table

We can always save the result of a query to a table

```
CREATE TABLE mytable AS
    SELECT teacher.dept_name, id, name, salary, building, budget
    FROM teacher, department
    WHERE teacher.dept_name = department.dept_name
```



mytable

dept_name	*id*	*name*	*salary*	*building*	*budget*
Comp. Sci.	10101	Srinivasan	65000.00	Taylor	100000.00
Finance	12121	Wu	90000.00	Painter	120000.00
Music	15151	Mozart	40000.00	Packard	80000.00
Physics	22222	Einstein	95000.00	Watson	70000.00
History	32343	El Said	60000.00	Painter	50000.00
Physics	33456	Gold	87000.00	Watson	70000.00
Comp. Sci.	45565	Katz	75000.00	Taylor	100000.00
History	58583	Califieri	62000.00	Painter	50000.00
Finance	76543	Singh	80000.00	Painter	120000.00
Biology	76766	Crick	72000.00	Watson	90000.00
Comp. Sci.	83821	Brandt	92000.00	Taylor	100000.00
Elec. Eng.	98345	Kim	80000.00	Taylor	85000.00

Saving Query to Table

```
CREATE TABLE mytable AS
    SELECT teacher.dept_name, id, name, salary, building, budget
    FROM teacher, department
    WHERE teacher.dept_name = department.dept_name
```

From its creation onwards, completely independent of the original table.

even if teacher or department changes, mytable remains the same

```
DELETE FROM teacher WHERE salary >= 80000
```

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
15151	Mozart	Music	40000.00
32343	El Said	History	60000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76766	Crick	Biology	72000.00

teacher changed . . .

mytable remains the same!

mytable

dept_name	*id*	*name*	*salary*	*building*	*budget*
Comp. Sci.	10101	Srinivasan	65000.00	Taylor	100000.00
Finance	12121	Wu	90000.00	Painter	120000.00
Music	15151	Mozart	40000.00	Packard	80000.00
Physics	22222	Einstein	95000.00	Watson	70000.00
History	32343	El Said	60000.00	Painter	50000.00
Physics	33456	Gold	87000.00	Watson	70000.00
Comp. Sci.	45565	Katz	75000.00	Taylor	100000.00
History	58583	Califieri	62000.00	Painter	50000.00
Finance	76543	Singh	80000.00	Painter	120000.00
Biology	76766	Crick	72000.00	Watson	90000.00
Comp. Sci.	83821	Brandt	92000.00	Taylor	100000.00
Elec. Eng.	98345	Kim	80000.00	Taylor	85000.00

in the full university schema, this DELETE would be **blocked** by foreign keys from teaches / advisor (recall **Error 2**), unless CASCADE or SET NULL is used.

Views

A view : a saved table constructed by a query, but ...

the view remains linked to the original tables ...

important: a view stores a query (in the form of a table)!

```
CREATE VIEW myview AS
    SELECT teacher.dept_name, id, name, salary, building, budget
    FROM teacher, department
    WHERE teacher.dept_name = department.dept_name
```

myview

dept_name	*id*	*name*	*salary*	*building*	*budget*
Comp. Sci.	10101	Srinivasan	65000.00	Taylor	100000.00
Finance	12121	Wu	90000.00	Painter	120000.00
Music	15151	Mozart	40000.00	Packard	80000.00
Physics	22222	Einstein	95000.00	Watson	70000.00
History	32343	El Said	60000.00	Painter	50000.00
Physics	33456	Gold	87000.00	Watson	70000.00
Comp. Sci.	45565	Katz	75000.00	Taylor	100000.00
History	58583	Califieri	62000.00	Painter	50000.00
Finance	76543	Singh	80000.00	Painter	120000.00
Biology	76766	Crick	72000.00	Watson	90000.00
Comp. Sci.	83821	Brandt	92000.00	Taylor	100000.00
Elec. Eng.	98345	Kim	80000.00	Taylor	85000.00

Views

```
CREATE VIEW myview AS
    SELECT teacher.dept_name, id, name, salary, building, budget
    FROM teacher, department
    WHERE teacher.dept_name = department.dept_name
```

From its creation onwards, always linked to the original tables.

if teacher or department change, myview changes

```
DELETE FROM teacher WHERE salary >= 80000
```

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
15151	Mozart	Music	40000.00
32343	El Said	History	60000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76766	Crick	Biology	72000.00

teacher changed!

myview

dept_name	*id*	*name*	*salary*	*building*	*budget*
Comp. Sci.	10101	Srinivasan	65000.00	Taylor	100000.00
Music	15151	Mozart	40000.00	Packard	80000.00
History	32343	El Said	60000.00	Painter	50000.00
Comp. Sci.	45565	Katz	75000.00	Taylor	100000.00
History	58583	Califieri	62000.00	Painter	50000.00
Biology	76766	Crick	72000.00	Watson	90000.00

myview automatically changed also!

VIEWS

convenient: view only the relevant attributes from a lot of data

maintenance: write a query **once** and reuse its **definition** everywhere
The view is recomputed at each use; MATERIALIZED VIEW to store results

security, privacy: decide what attributes to be seen by certain users

salary kept in the database, but view does not show it to student users

can delete them by

```
DROP VIEW myview;
```

however, updating using views is more complicated